

C o n t r o l l e r

Controller

Principle

First object behind the UI

DEFINITION

Hand a system event to a non-UI object representing the system, a use case, or a root — not the UI widget.

Controller decides who first handles a system operation — a request arriving from outside the software. The answer is a non-UI object: one representing the whole system (a facade controller), a specific use case or session, or a root domain object. The UI widget captures the event, then immediately hands it to this controller, which coordinates the domain.

WHY IT MATTERS

Routing input through a dedicated controller keeps domain logic out of buttons and HTTP handlers, so the same use case can be driven by web, CLI, tests, or a queue. It gives one place to manage a use case's flow and transaction boundary, and it stops the UI from coupling directly to domain internals.

— How it works

Controller	The first non-UI object to receive a system operation. Represents the system, a use case, or a root — and only coordinates.
UI	Captures the raw event (click, request) and delegates inward without running business rules itself.
Domain	The services and entities the controller drives to actually fulfil the operation.

HOW TO APPLY

1. Spot the system operation — the event crossing into your software.
2. Pick a non-UI handler: a use-case controller for big systems, a facade controller for small ones.
3. Let the UI call only that controller — no domain logic in the widget.
4. Keep the controller thin: it routes and coordinates; rules live in services and entities.

LITMUS TEST

If you deleted the UI, would this logic still be needed? If yes, it belongs behind a controller, not inside the click handler.

— Smell → fix

A UI widget runs domain logic directly — untestable without the UI, tied to one delivery:

```
class CheckoutButton {  
  onClick(dto: PlaceDto) { // a UI widget  
    const order = new Order(dto) // domain work in the UI  
    this.db.save(order)  
  }  
}
```

↓ DELEGATE INTO THE DOMAIN

```
@Controller('orders')  
class OrdersController { // not the UI – delegates inward  
  @Inject(OrderService) private orders!: OrderService  
  place(dto: PlaceDto) { return this.orders.place(dto) }  
}
```

The controller only routes; OrderService holds the use case — now reused by web, CLI, and tests while the UI stays thin.

USE WHEN

- ✓ Routing input from the UI into the domain
- ✓ Use-case / session coordination

AVOID WHEN

- ▲ A bloated controller doing domain work itself — delegate it

— Trade-offs & pitfalls

PROS

- + Keeps the UI thin
- + Domain logic reused across UIs

CONS

- Risk of a god controller
- Over-centralization

COMMON PITFALLS

- ▲ The "god controller" that accumulates business logic instead of delegating — split it by use case.
- ▲ One bloated facade controller for a large system, when per-use-case controllers would stay cohesive.
- ▲ Leaking domain work back into the UI because the controller felt like "extra indirection".

WHERE YOU SEE IT

- A NestJS @Controller mapping a route to a service call — HTTP boundary, no business rules.
- An application / use-case handler (PlaceOrderHandler) coordinating one scenario end to end.
- MVC's Controller and MVP's Presenter — concrete realizations of this principle.

SEE ALSO

MVC	GRASP names the principle; MVC's Controller is one concrete realization
Pure Fabrication	a use-case controller is usually a fabricated, non-domain class
Low Coupling	keeps the UI from depending directly on domain internals
Indirection	the controller is a middleman between delivery and domain

— Interview & sources

GRASP Controller vs MVC Controller?

Same idea, different scope: GRASP states the principle (a non-UI object handles system events); MVC's Controller is one realization of it.

Facade controller vs use-case controller?

A facade (system) controller handles all events of a subsystem — simple but bloats. A use-case (session) controller handles one scenario — better cohesion for large systems.

How do you avoid a "god controller"?

Keep it thin: only receive and delegate. Domain rules live in services and entities; if the controller starts deciding, push that logic inward.

Should controllers contain business logic?

No — a controller coordinates: it receives the operation, calls the right domain services, and returns a result. Rules belong in entities and services so they stay reusable and testable without the delivery layer.

REMEMBER

The first object to receive a system event should not be the UI.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Controller

Martin Fowler — "Patterns of Enterprise Application Architecture" (2002): Input Controller